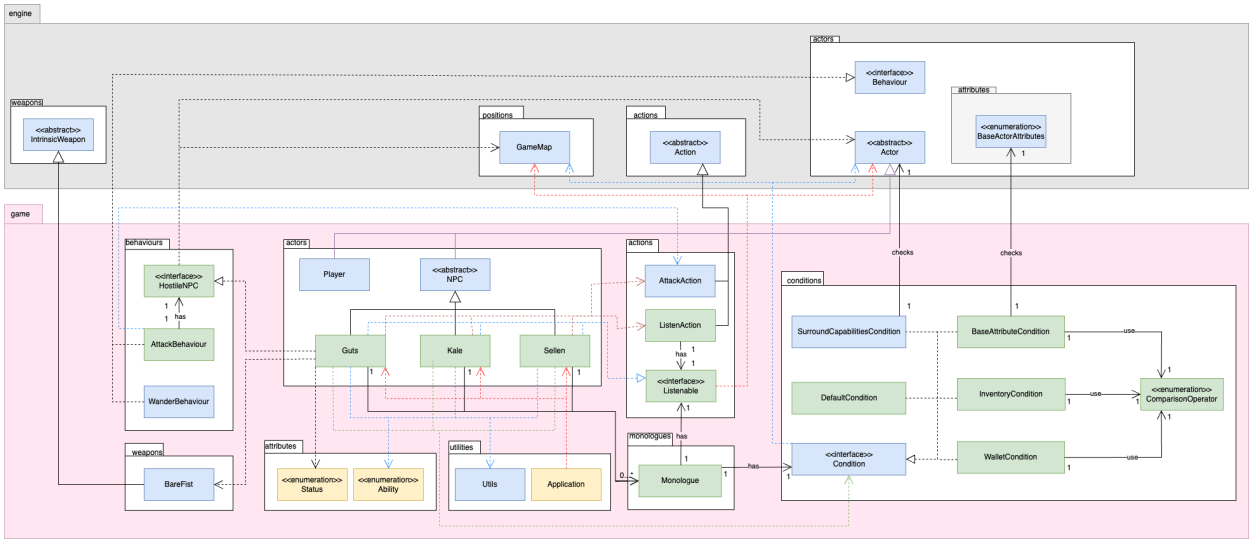
Design Documentation

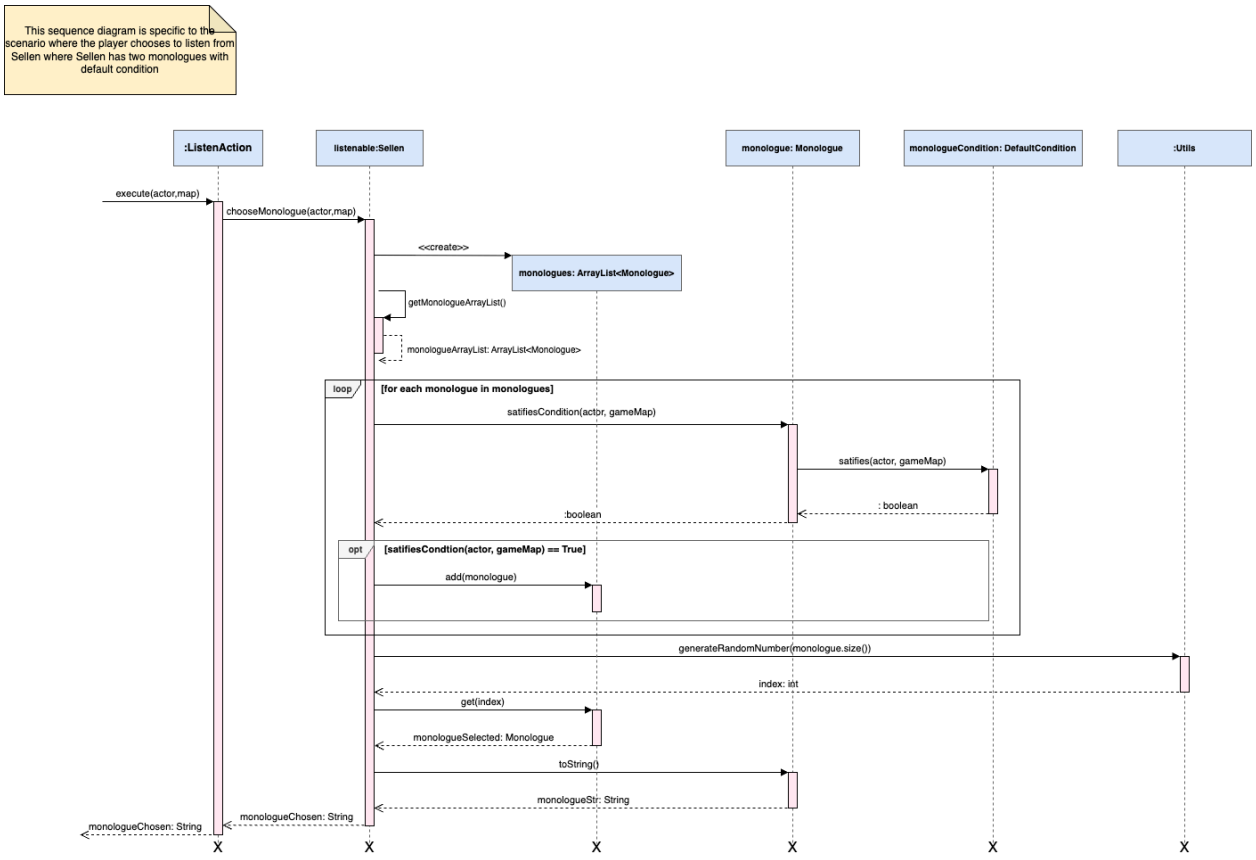
Req 3 - The People of the Valley

UML diagram:



Existing Class (include previous req)	Blue
Modified Class	Yellow
New Class	Green

Sequence Diagram:



Design Rationale:

A. Implementation of Kale, Sellen and Guts

Design 1: Create concrete classes representing them each extending Actor abstract class.

Pros	Cons
Simple design, easy to implement each specified functionality without inheriting any functionality.	Code redundancy, all classes have behaviours in the game.
	Not fully utilizing existing design, 'NPC' abstract class defined the base shared methods and attributes.

Design 2: Create concrete classes representing them each extending existing 'NPC' abstract class.

Pros	Cons
Reduce code redundancy, extend common logic as NPC in abstract class	Tighter coupling for deeper inheritance level
Flexibility in future extension, new shared logic can be added in abstract class, while specified logic can be implemented inside specified classes.	

B. Added HostileNPC interface

Pros	Cons
Promotes high cohesion. Any HostileNPC can implement their own attacking behaviour related logic.	High cost for interface refactoring if there are many Hostile NPC.
Promotes low coupling. Reduce dependencies where any related to HostileNPC only depend on this interface.	

C. Implementation of Listenable interface

Pros	Cons
Promotes high cohesion. Any Listenable can define their own Monologue and choosing logic.	High cost for interface refactoring if there are a lot of Listenable.
Promotes low coupling. Reduce dependencies where classes need a Listenable only depend on Listenable.	

D. Implementation of ListenAction class extending Action abstract class

Pros	Cons
Promotes encapsulation. Encapsulates the execution related logic.	High cost for Action class refactoring if major modification is done in Action abstract class.
Promote extension, allowing any new NPC with Listen feature to be introduced by adding ListenAction.	

E. Implementation of AttackBehaviour implementing Behaviour interface

Pros	Cons
Follows general way of other behaviours is implemented	High cost for any modification made to the Behaviour interface
Promote code reusability. Any hostile NPC can add this behaviour to the behaviour list only.	

F. Checking capabilities of certain entities perform certain actions

Validations involves checking capabilities:

- Guts can attack moving actors
- Player can listen to several NPC

Design 1: Use `instanceOf()` to check the entity type

Pros	**Cons**
Simple to implement	Poor extensibility when more entities are expected to pass these validations
	Violating OCP and LSP when overusing `instanceOf()`

Design 2: Add enum MOVE_ABILITY (all current actors), LISTEN ABILITY(Player), HOSTILE_TO_ENEMY(Guts)

Pros	**Cons**
Introduce extensibility when more entities are expected to pass these validations	Increase the validation code length when overusing enum value

G. Implementation of Monologue concrete class

Pros	**Cons**
Introduce encapsulation by encapsulating all details and logic of a monologue in a class	May introduce unnecessary complexity when the monologue details can be defined in actor classes

H. Create condition concrete classes to implementing Condition interface

- BaseAttributeCondition
- WalletCondition
- DefaultCondition
- InventoryCondition

Pros	Cons
Promote high cohesion, where each class focuses on single responsibility instead of having multiple conditions being grouped together	More class to maintain, debugging complexity may increase.
Modular code, these classes can be reused by classes which need condition checking.	

I. Add magic numbers and strings to be constant

Introduced *NAME, SYMBOL, HITPOINT* constants in Kale, Sellen and Guts class to reduce the level of connascence of meaning. And added RUNE_THRESHOLD as constant variable in Kale to reduce connascence of meaning, allowing easy maintainability when rune threshold needs to be modified in the future.

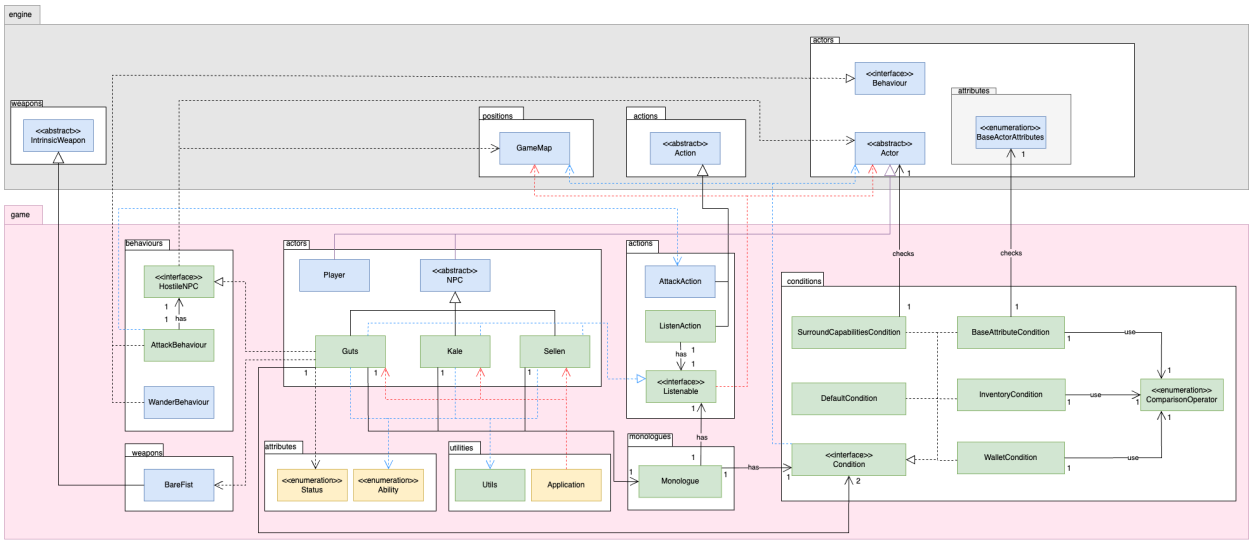


Figure above shows the UML diagram which follows part A (Design 2), B, C, D, E, F (Design 2), G, H, I.
Created a Listenable interface with addMonologue and chooseMonologue method. This allows each Listenable to add their own defined Monologue and to have different logic for choosing Monologue to speak (SRP). Moreover, we can easily add more NPC that can be listened to by simply implementing this interface (OCP). And, it can be substituted into anywhere where a Listenable is expected and anywhere which needs a Listenable will only depend on this interface (LSP & DIP).

Created a ListenAction class which extends from Action abstract class. Any NPC with the ability to speak their monologue can perform this action simply by adding this action into the ActionList (OCP). ListenAction class is encapsulated and only responsible for executing the necessary method for a Listenable NPC to speak to an actor (SRP).

Created a stand alone Monologue concrete class. Each object of this class only represents a single Monologue with the message, speaker and condition (SRP). It is only responsible for executing logic for when to display the Monologue but not handle the detailed logic.

Create concrete classes that implement Condition interface such as BaseAttributeCondition, InventoryCondition etc. This allows every part of the code which involves using the same conditional logic can simply reuse the same class object (DRY). These classes can easily substitute in any place that needs Condition (LSP). Each class can define their own logic about their own conditional behavior (SRP).

Created HostileNPC interface to allow each HostileNPC to define their own logic of condition to attack. Each class that implements HostileNPC will have their own defined attack condition encapsulated in their class (SRP) and we can easily introduce more hostile NPC by implementing this interface (OCP) without modifying other classes. Anything that needs HostileNPC will only need to depend on this interface only (DIP).

Created AttackBehaviour which implements Behaviour interface and extends Action class. Any HostileNPC can simply add this Behaviour to attack in game without modifying the existing logic (OCP). It also can be substituted anywhere which expects a AttackBehaviour (LSP).

Kale, Sellen and Guts depend on enumeration class ‘Ability’ and Guts depends on the enumeration class ‘Status’. It acts as an indicator to allow certain actions and avoid down-casting to check for each instance type to determine whether they are allowed to perform certain actions. Hence, making it easier for future extension in the game, where new classes that can perform the action can execute the action easily (OCP). And, classes will depend on the enumeration class instead of concrete classes (DIP).